



MICROSOFT
STUDENT COMMUNITY
SRM AP



SRM
UNIVERSITY AP
— Andhra Pradesh

SYNORA

PROBLEM STATEMENT



SYNORA

Track Problem Statements & Mentor Briefing Guide

Industry Mentor & Judge Edition | AI/ML • Cyber Security • Full Stack Engineering • Web3 & Blockchain

Welcome Note & Executive Overview for Industry Mentors & Judges

Welcome to SYNORA! This document provides a complete guide to all hackathon problem statements across four core technical tracks: AI/ML, Cyber Security, Full Stack Engineering, and Web3 / Blockchain. As industry leaders and mentors, your role is pivotal in guiding participants toward production-grade engineering, architectural rigor, scalability, and robust user experience. Each problem statement below outlines the challenge context, core functional and hard technical requirements, suggested technology stacks, and specific evaluation criteria.

Evaluation Framework & Scoring Guidelines

When reviewing participant solutions, please evaluate across the criteria below:

Criteria	Weight	What Judges Evaluate
Problem Understanding & Relevance	15%	How clearly the team identifies the problem and whether the solution genuinely addresses it
Innovation & Creativity	20%	Originality, creativity, unique approach, and differentiation
Technical Implementation	30%	Technical complexity, functionality, architecture, APIs, AI/ML, database, integrations, etc.
Impact & Feasibility	15%	Real-world usefulness, target users, feasibility, scalability, and potential impact
UI/UX & Design	10%	Usability, accessibility, interface quality, user flow, and overall experience
Presentation & Demo	10%	Clarity, communication, live demonstration, and ability to answer questions
Total	100%	

TRACK 1

AI / Machine Learning

01 End-to-End Career & Livelihood Guidance Agent

Merges: Career Guidance & Interview Readiness Agent + Inclusive Access Career & Livelihood Agent

CONTEXT & BUSINESS IMPACT

Traditional career counseling and job matching platforms suffer from significant fragmentation. Job seekers from diverse socioeconomic backgrounds, ranging from university graduates aiming for technical roles to rural workers navigating local gig markets, face severe information asymmetry. The objective is to build a unified, intelligent agent capable of delivering personal, adaptive, and culturally accessible career progression paths.

THE CHALLENGE

Build a unified career-guidance platform prototype serving two distinct personas via a shared core matching engine:

- Students: Skill assessment, adaptive roadmap, and targeted interview feedback.
- Rural Youth / Informal Workers: Vernacular-language government scheme awareness and local job leads.

KEY FUNCTIONAL & TECHNICAL REQUIREMENTS

- Adaptive Roadmap: Demonstrate before/after roadmap changes triggered by simulated skill updates.
- Live Data Sources: Job listings and government schemes must be fetched dynamically via live APIs or scraping, with no hardcoded lists.
- Actionable Interview Feedback: Highly specific feedback, avoiding generic praise.
- Vernacular & Voice I/O: End-to-end support for at least one regional language with first-class voice input and output.

Shared Architectural Engine: Core matching engine must be shared across both UIs and verified in code.

SUGGESTED ARCHITECTURE & TECH STACK

Open LLMs (Llama 3 / Mixtral), Vector DB (Pinecone / Qdrant), Job APIs (Adzuna / LinkedIn Scraped Data), Speech & Translation (Whisper / IndicTrans2), Frontend (React / Next.js / WhatsApp API via Twilio).

MENTOR EVALUATION GUIDELINES

Assess system architecture for shared core component reuse across personas, dynamic graph/vector updates in roadmaps, precision of RAG retrieval for regional schemes, voice latency, and actionable depth of mock interview feedback.

02 Holistic Health & Wellness Agent

Merges: Chronic Health Monitoring & Symptom Triage Agent + Personal Wellness Coach (Fitness + Nutrition)

THE CHALLENGE

Construct a comprehensive health agent combining proactive fitness/nutrition coaching with safe medical triage and chronic condition tracking.

CONTEXT & BUSINESS IMPACT

Digital health applications frequently operate in silos, isolating lifestyle/wellness coaching from clinical triage and chronic disease management. A unified agent must bridge this gap while enforcing clinical safety boundaries, non-diagnostic guardrails, and real-time biometric anomaly detection.

KEY FUNCTIONAL & TECHNICAL REQUIREMENTS

- Clinical Safety & Escalation: Explicit urgency framing; non-diagnostic outputs; clear referral triggers.
- Intent Routing: Dynamic routing between wellness coaching and clinical triage on ambiguous inputs.
- Multimodal Meal Recognition: Real food photo analysis via Vision LLMs/VLMs.
- Constraint Enforcement: Health condition parameters must override general wellness recommendations.

Sensor Anomaly Detection: Real-time anomaly detection layer operating on synthetic wearable sensor data.

SUGGESTED ARCHITECTURE & TECH STACK

Open LLMs with Clinical RAG (BioMistral / Med-PaLM / PubMed Retriever), Vision-Language Models (LLaVA / GPT-4V), Wearable Stream Processor (Kafka / Flink / Python Anomaly Detector), React Native / Flutter.

MENTOR EVALUATION GUIDELINES

Verify clinical safety rails (refusal to diagnose, mandatory escalation triggers), deterministic intent classification, accuracy of macro estimation from meal imagery, and latency in anomaly alert generation from biometric streams.

03 Financial & Entrepreneurial Empowerment Agent

Merges: Startup Funding & Blueprint Agent + Financial Inclusion & Informal Commerce Agent

THE CHALLENGE

Deliver an intelligent assistant guiding micro-entrepreneurs from basic financial security (fraud prevention, QR generation) up to startup acceleration (blueprints, grant matching).

CONTEXT & BUSINESS IMPACT

Micro-entrepreneurs and early-stage founders face huge operational hurdles, from financial literacy and payment fraud vulnerability to navigating complex venture capital or government grant ecosystems. A single conversational lifecycle agent can democratize financial access and venture building.

KEY FUNCTIONAL & TECHNICAL REQUIREMENTS

- Real-world Fraud Prevention: Current UPI scam patterns and actionable prevention guidance.
- Actionable Onboarding: Real dynamic QR code generation for informal merchants.
- Live Grant/Funding Aggregation: Scraped/aggregated live funding data from 2–3 authentic sources.
- Tailored Applications: Blueprint details dynamically populate grant application drafts without silent broad matching.

Seamless Conversational Escalation: Conversational transition from fraud prevention to formal grant applying within one session.

SUGGESTED ARCHITECTURE & TECH STACK

Open LLMs + Function Calling / Tool Use, Scrapy / Playwright for grant aggregation, Dynamic QR Generation libraries (qrcode / libqrencode), IndicTrans2 translation, FastAPI / Node.js backend.

MENTOR EVALUATION GUIDELINES

Focus on live grant data freshness, accuracy of auto-generated business plan artifacts, prevention of hallucinated grant deadlines, seamless conversational state management, and localized financial literacy UX.

TRACK 2

Cyber Security

01 JWT & Auth Token Attack Surface Audit

THE CHALLENGE

Develop an authentication service containing a genuine real-world token vulnerability alongside a companion auditing tool that detects and flags the flaw.

CONTEXT & RISK PROFILE

Modern cloud microservices rely heavily on stateless token-based authentication. Implementation flaws in JWT signature verifications, token parsing libraries, or claim assertions create catastrophic attack vectors permitting full account takeover or privilege escalation across distributed systems.

KEY TECHNICAL & EXPLOITATION REQUIREMENTS

- Exploitable Auth System: Include flaws such as alg: none, weak secrets, missing expiry, or bad audience/issuer verification.
- Working Exploit: Live demonstration of exploiting the flaw.
- Generalized Audit Tool: The auditor must successfully identify flaws in a completely separate, unseen target JWT implementation.

SUGGESTED ARCHITECTURE & TECH STACK

Vulnerable Node.js / Go auth server, Python (requests, PyJWT, cryptography), CLI auditor interface, Dockerized target suite.

MENTOR EVALUATION GUIDELINES

Evaluate exploit reliability during live execution, ability of the auditor to generalize across unseen implementations, structured remediation report clarity, and zero false-positive rates on hardened JWT implementations.

02 Tamper Attack Prevention System

THE CHALLENGE

Build a real-time defense framework detecting dynamic physical/digital tampering on application processes and binary integrity.

CONTEXT & OPERATIONAL RISK

Enterprise endpoints and runtime environments are continuously targeted by binary patch attacks, dynamic memory injection, and unauthorized file modification. Real-time integrity monitoring and automated incident mitigation are crucial for maintaining system trust in high-stakes environments.

KEY TECHNICAL & OPERATIONAL REQUIREMENTS

- Continuous Auditing: Hash verification, signature validation, or memory layout auditing.
- Real-Time Breach Detection: Instant detection of injection or dynamic file/configuration changes.
- Automated Mitigation: Session termination, process isolation, key revocation, and alert generation.

Tamper-Evident Logs: Secure, immutable logging mechanism.

SUGGESTED ARCHITECTURE & TECH STACK

Rust / C++ (low-level hooks / OS API monitoring), eBPF (Linux kernel telemetry), SQLite / InfluxDB, React + WebSockets security operational dashboard.

MENTOR EVALUATION GUIDELINES

Measure sub-second response time to memory/binary injection, cryptographic verification speed of log stores, low CPU/RAM overhead of the agent, and efficacy of automated isolation mechanics.

03 Red Team / Blue Team Live Exercise Platform

THE CHALLENGE

Create a vulnerable-by-design web app coupled with a live incident response monitoring dashboard.

CONTEXT & OPERATIONAL RISK

Security Operations Center (SOC) analysts and incident response teams require automated visual telemetry to correlate disparate log streams during active adversary intrusion. A closed-loop exercise platform demonstrates real-time visibility and threat containment tactics.

KEY TECHNICAL & OPERATIONAL REQUIREMENTS

- Vulnerable Application: Incorporate at least 2 distinct web vulnerabilities (e.g., SQLi, IDOR, Stored XSS).
- Live Detection Dashboard: Real-time ingestion and alert generation during active attacks.

Incident Timeline: Reconstruct complete attack vectors chronologically from captured logs.

SUGGESTED ARCHITECTURE & TECH STACK

Vulnerable Application (Python Flask / Express.js), Ingestion Engine (Elasticsearch / Fluentd / Custom Node Stream), Frontend SOC Dashboard (React / Tailwind CSS / Chart.js / Socket.io).

MENTOR EVALUATION GUIDELINES

Evaluate accuracy of attack vector reconstruction, ingestion throughput under multi-threaded exploit execution, UI response time, and clarity of automated threat containment alerts.

TRACK 3

Full Stack Engineering

01 Real-Time Collaborative Document Editor

THE CHALLENGE

Engineer a Google Docs-style rich text editor enabling simultaneous multi-user editing without data loss or race conditions.

SYSTEM COMPLEXITY & ARCHITECTURE

Collaborative editing engines represent one of the most challenging distributed systems paradigms. Synchronizing real-time mutations across latent, unreliable networks demands fault-tolerant data structures, low-latency state propagation, and robust local persistence models.

KEY DISTRIBUTED SYSTEMS REQUIREMENTS

- Conflict Resolution: Mandatory CRDT (Yjs, Automerge) or OT implementation (Last-Write-Wins is disallowed).
- Live Presence: Synchronized multi-user cursors and active selection highlighting.

Persistence & Recovery: Full recovery of document state following mid-session network reconnections.

SUGGESTED TECH STACK

CRDT Core (Yjs / Automerge / Loro), Networking (y-websocket / WebRTC), Frontend (TipTap / Slate / Quill + React / Next.js), Persistence (PostgreSQL / Redis / IndexedDB).

MENTOR EVALUATION GUIDELINES

Evaluate state consistency under concurrent network partitioning simulations, latency of cursor/selection synchronization across clients, memory footprint of the CRDT structure over long document lifetimes, and offline sync performance.

02 Real-Time Bidding / Auction System Under Load

THE CHALLENGE

Architect a high-throughput transactional auction system ensuring valid bid serialization under heavy concurrent loads.

SYSTEM COMPLEXITY & ARCHITECTURE

High-concurrency financial trading and bidding platforms must enforce absolute data integrity while handling thousands of sub-millisecond transaction attempts per second. Preventing race conditions, ghost bids, and double-allocation requires resilient database locking and streaming architecture.

KEY DISTRIBUTED SYSTEMS REQUIREMENTS

- Bid Serialization: Reject out-of-order or invalid bids lower than current highest during race conditions.
- Low-Latency Broadcasts: Real-time push of updated prices to all connected clients.
- Live Stress Demo: Live execution under concurrent submission spikes.

SUGGESTED TECH STACK

Go / Rust / Node.js backend, Transactional Storage (PostgreSQL with Serializable Isolation / Redis Lua Scripts), Pub/Sub (Redis Pub/Sub / Apache Kafka), Load Testing (k6 / Locust).

MENTOR EVALUATION GUIDELINES

Verify zero invalid or out-of-order bid acceptances under simulated multi-client stress tests (e.g., 5,000 req/sec), real-time delivery latency of highest-bid updates to UI clients, and graceful degradation under DB lock contention.

03 Distributed Task Scheduler & Worker System

THE CHALLENGE

Build a fault-tolerant, scalable background job queue and worker execution engine.

SYSTEM COMPLEXITY & ARCHITECTURE

Distributed asynchronous execution systems power large-scale enterprise workflows. Designing a resilient scheduler requires solving distributed lock management, node failure detection, worker re-balancing, and idempotent execution guarantees under high message volume.

KEY DISTRIBUTED SYSTEMS REQUIREMENTS

- Execution Semantics: At-least-once task execution with support for delayed/scheduled jobs.
- Failure Handling: Dead-Letter Queue (DLQ) re-processing and worker heartbeat detection with automatic task reassignment.

Observability Dashboard: Real-time metrics on queue depth, worker health, and task status.

SUGGESTED TECH STACK

Go / Python / Node.js, Message Broker (Redis / RabbitMQ / NATS), Relational Persistence (PostgreSQL), Frontend Telemetry (Next.js + WebSockets / SSE).

MENTOR EVALUATION GUIDELINES

Test live failover by killing worker processes mid-task execution, assess queue starvation prevention during heavy loads, measure scheduling precision for delayed tasks, and review telemetry dashboard responsiveness.

TRACK 4

Web3 & Blockchain

01 Decentralized Escrow & Cross-Border Settlement Protocol

THE CHALLENGE

Develop a non-custodial smart contract escrow protocol for trustless cross-border settlements using stablecoins.

CONTEXT & PROTOCOL ARCHITECTURE

Traditional cross-border settlements suffer from high intermediary fees, multi-day delays, and counterparty risks. Non-custodial escrow smart contracts facilitate automated, trustless settlement conditioned on off-chain/on-chain proof validation while minimizing capital lockup risks.

KEY SMART CONTRACT & PROTOCOL REQUIREMENTS

- Escrow Release: Programmatic funds release via multi-signature consensus or oracle verification.
- Safety Mechanics: Built-in timeout and refund arbitration logic.

dApp Interface: Clean UI illustrating real-time contract state transitions and token transfers.

SUGGESTED TECH STACK

Smart Contracts (Solidity / Vyper + OpenZeppelin), Development Framework (Foundry / Hardhat), Oracles (Chainlink / API3), Web3 Frontend (Viem / Wagmi / RainbowKit + React).

MENTOR EVALUATION GUIDELINES

Audit smart contracts for reentrancy, check-effects-interactions pattern compliance, gas efficiency under high-frequency executions, correct dispute/timeout logic, and intuitive Web3 wallet UX.

02 Zero-Knowledge Proof Credentials & Identity Verifier

THE CHALLENGE

Construct a privacy-focused identity verification system utilizing ZKPs to validate compliance claims without disclosing underlying sensitive data.

CONTEXT & PROTOCOL ARCHITECTURE

Identity verification in Web3 and corporate systems typically mandates revealing sensitive personally identifiable information (PII). Zero-Knowledge Cryptography empowers users to mathematically prove compliance (e.g., age, accredited investor status, solvency) without leaking underlying raw credentials.

KEY CRYPTOGRAPHIC & PROTOCOL REQUIREMENTS

- Proof Generation: Generate valid zk-SNARK / zk-STARK proofs for compliance verification (e.g., age, balance threshold).
- Verification: Verify proofs client-side or on-chain without exposing raw data.

User Workflow: Full UI pipeline for credential issuance and proof verification.

SUGGESTED TECH STACK

ZK Circuits (Circom / Noir / SnarkJS), On-Chain Verification (Solidity generated verifiers), Frontend (React / Next.js / WASM for in-browser client proof generation).

MENTOR EVALUATION GUIDELINES

Verify mathematical zero-knowledge properties (ensuring zero private data leak in public signals), measure client-side proof generation speed, evaluate on-chain verification gas costs, and review credential issuance UX.

03 Dynamic NFT Loyalty & Token-Gated Community Platform

THE CHALLENGE

Build an evolving NFT (dNFT) platform where asset metadata updates automatically in response to user actions or event check-ins, unlocking gated features.

CONTEXT & PROTOCOL ARCHITECTURE

Static digital assets lack ongoing utility and real-time engagement incentives. Dynamic NFTs (dNFTs) leverage dynamic metadata pipelines triggered by automated off-chain actions or oracle automation, transforming digital credentials into living, verifiable identity cards.

KEY CRYPTOGRAPHIC & PROTOCOL REQUIREMENTS

- Metadata Evolution: ERC-721 / ERC-1155 smart contract supporting authorized dynamic metadata updates.
- Token Gating: Access control verification unlocking restricted content based on NFT state.

Event Trigger: Interactive check-in system driving state evolution.

SUGGESTED TECH STACK

Smart Contracts (ERC-721 / ERC-1155 in Solidity), Metadata Storage (IPFS / Arweave / Dynamic JSON Relay), Automation (Chainlink Automation / Gelato Network), Frontend (React / Viem / Next.js).

MENTOR EVALUATION GUIDELINES

Assess dynamic metadata update reliability, security of state-change authorization, token-gating verification latency, gas efficiency during batch check-in events, and overall end-to-end dApp flow.

TRACK 5

Internet of Things (IoT)

01 Smart Industrial Edge Telemetry & Anomaly Detection

THE CHALLENGE

Design and implement an end-to-end IoT monitoring platform that ingests high-frequency edge sensor data, performs local real-time anomaly detection, and triggers automated field safety alerts.

CONTEXT & OPERATIONAL IMPACT

Industrial manufacturing plants rely heavily on continuous sensor streams to monitor machine health, vibration, temperature, and power draw. Unplanned equipment downtime and unflagged anomalies cause severe financial loss and safety hazards. An edge-aware IoT infrastructure ensures sub-second anomaly detection and resilient operation even during intermittent network connectivity.

KEY FUNCTIONAL & HARD TECHNICAL REQUIREMENTS

- Edge Ingestion & Processing: Real-time telemetry parsing from microcontrollers or edge nodes, easily implemented online via online simulators (such as Wokwi or Tinkercad) or physically using available electronic components via light protocols (MQTT / CoAP).
- Edge Anomaly Engine: Lightweight local ML model or threshold rule-engine running at the edge or gateway level to flag critical operational spikes.
- Offline Store-and-Forward: Resilient local buffering during network outages with seamless sync to the cloud on connection restoration.
- Central Operational Dashboard: Real-time telemetry dashboard featuring interactive time-series plots, active alerts, and remote device management.

SUGGESTED ARCHITECTURE & TECH STACK

ESP32 / Raspberry Pi / Wokwi Simulator, MQTT Broker (Mosquitto / EMQX), Time-Series Database (InfluxDB / TimescaleDB), Backend (Node.js / Python FastAPI), Frontend Dashboard (React / Grafana / Tailwind CSS).

MENTOR EVALUATION GUIDELINES

Evaluate sub-second anomaly alert latency, store-and-forward reliability during simulated network drops, edge resource utilization (CPU/RAM footprint), and overall responsiveness of the central telemetry dashboard.

02 Smart Home Energy & Appliance Monitor

THE CHALLENGE

Build an IoT-based home energy monitoring platform that captures live electrical current/voltage data, estimates appliance-level usage, and alerts users to inefficient energy consumption patterns.

CONTEXT & OPERATIONAL IMPACT

Rising utility costs and sustainability concerns require intelligent visibility into household energy consumption. Traditional smart meters provide aggregate billing data without granular appliance breakdowns. Real-time power signature tracking enables proactive energy conservation, peak-load shaving, and automated fault detection for household electronics.

KEY FUNCTIONAL & HARD TECHNICAL REQUIREMENTS

- Real-Time Power Sensing: Sample current/voltage telemetry at regular intervals using online hardware simulators (such as Wokwi or Tinkercad) or physical electronic sensors.
- Load Classification: Basic rule-based or lightweight ML model to distinguish major load types (e.g., HVAC, lighting, refrigeration).
- Budgeting & Threshold Alerts: Push notifications when daily or monthly power usage exceeds customizable thresholds.
- Energy Dashboard: Interactive UI showing live power draw (Watts), historical usage trends, and estimated cost breakdown.

SUGGESTED ARCHITECTURE & TECH STACK

ESP32 / Arduino / Wokwi Simulator, Current Sensor (SCT-013), Protocol (MQTT / HTTP), Backend (Node.js / Python FastAPI), DB (InfluxDB / SQLite), Mobile/Web App (React / Flutter).

MENTOR EVALUATION GUIDELINES

Evaluate telemetry reporting accuracy, power estimation precision, responsiveness of budget threshold alerts, and clarity of energy visualization components.

03 Connected Agriculture & Automated Irrigation System

THE CHALLENGE

Develop an automated precision irrigation network that monitors soil moisture, ambient temperature, and weather forecasts to dynamically trigger water valve actuation.

CONTEXT & OPERATIONAL IMPACT

Agricultural water waste and crop failure due to irregular irrigation remain critical challenges for modern farming. By replacing manual scheduling with closed-loop sensor-driven feedback and predictive weather integration, automated irrigation systems conserve water resources while maximizing crop yield.

KEY FUNCTIONAL & HARD TECHNICAL REQUIREMENTS

- Multi-Sensor Ingestion: Continuous monitoring of soil moisture, ambient temperature, and humidity levels, constructible either virtually via online simulators (such as Wokwi or Tinkercad) or physically with hardware components.
- Automated Actuation: Programmatic relay/valve control based on configurable soil moisture thresholds.
- Weather API Integration: Skip or delay automated watering cycles if rainfall is forecasted in the near term.
- Remote Override & Status: Farm management dashboard allowing manual irrigation overrides and historical soil data analysis.

SUGGESTED ARCHITECTURE & TECH STACK

ESP8266 / Raspberry Pi / Online Simulators (Wokwi, Tinkercad), Moisture & DHT22 Sensors, Relay Module, MQTT / HTTP, OpenWeatherMap API, Node.js / Python backend, React / Tailwind CSS dashboard.

MENTOR EVALUATION GUIDELINES

Assess closed-loop threshold control, logic for weather API integration override, manual override responsiveness, and end-to-end telemetry reliability.